Universidad Católica de Temuco

Laboratorio N°2

Expresiones Regulares con JFLAP

Teoría de la Computación

Dato	**Información**
Código curso	INFO1148
Integrantes	Matias Valenzuela Matias Vasquez
Profesor	Prof. M. Lévano
Fecha de entrega	11/10/2026

Índice

1. Introducción **2**

2. Pregunta 1: Interpretación y validación de una expresión regular **3**
2.1. Explicación de la expresión . . . 3
2.2. Conversión de la expresión regular a AFN . . . 3
2.3. Predicciones . . . 4
2.4. Ejecución en JFLAP y comparación . . . 4
2.5. Análisis de las cadenas `1101` y `0100` . . . 5

3. Pregunta 2: Diseño de una expresión regular **6**
3.1. Diseño de la expresión regular . . . 6
3.2. Justificación de la expresión . . . 6
3.3. AFN generado por el diseño planteado . . . 6
3.4. Resultados esperados y obtenidos en JFLAP . . . 7

4. Pregunta 3: Conversión ER → AFN → AFD **9**
4.1. Diseño de la expresión regular . . . 9
4.2. Conversión de la expresión regular a AFN . . . 9
4.3. Conversión de AFN a AFD . . . 10
4.4. Características de ambos autómatas . . . 10
4.5. Ejecución de cadenas en AFN y AFD . . . 11
4.6. ¿El AFN y el AFD reconocen lenguajes diferentes? . . . 12

5. Pregunta 4: Problema de diseño y validación **13**
5.1. Diseño de la expresión regular . . . 13
5.2. Estrategia utilizada . . . 13
5.3. Implementación en JFLAP . . . 13
5.4. Conversión ER → AFN → AFD . . . 14
5.5. Casos de prueba . . . 16
5.6. Reflexión . . . 17

6. Conclusión **18**

7. Archivos adjuntos **19**

1. Introducción

El propósito de este laboratorio es aplicar los conceptos de lenguajes regulares y expresiones regulares utilizando la herramienta JFLAP, relacionando una misma expresión regular con los autómatas finitos que reconocen su lenguaje. Se trabajó con la interpretación de una expresión dada, el diseño de expresiones a partir de restricciones sobre cadenas, la conversión de una expresión regular a un autómata finito no determinista (AFN) y la conversión de un AFN a un autómata finito determinista (AFD) mediante la construcción por subconjuntos. En cada pregunta los resultados se validaron experimentalmente ejecutando cadenas aceptadas y rechazadas en JFLAP, para comprobar que la expresión regular, el AFN y el AFD representan el mismo lenguaje.

2. Pregunta 1: Interpretación y validación de una expresión regular

Se considera el alfabeto $\Sigma = \{0, 1\}$ y la expresión regular:

$$(0 + 1)^*01$$

2.1. Explicación de la expresión

La expresión se puede leer por partes, de izquierda a derecha, como se muestra en la Tabla 1.

Tabla 1: Componentes de la expresión regular y su significado

Parte	Significado
$0 + 1$	Un `0` o un `1` (el símbolo + denota unión).
$(0 + 1)^*$	Cualquier cantidad de ceros y unos, incluida la cadena vacía λ.
01	Un `0` seguido de un `1`, de forma obligatoria.

Esta expresión representa el lenguaje de todas las cadenas sobre Σ que terminan en `01`. El prefijo $(0+1)^*$ puede ser cualquier cadena binaria, incluida la cadena vacía λ, mientras que el sufijo `01` obliga a que los dos últimos símbolos sean un `0` seguido de un `1`. Por lo tanto, la cadena más corta del lenguaje es `01`.

$$L = \{\, w \in \{0, 1\}^* \mid w \text{ termina en } 01 \,\}$$

2.2. Conversión de la expresión regular a AFN

La expresión se ingresó en JFLAP mediante la opción *Regular Expression* y se convirtió con *Convert* $\rightarrow$ *Convert to NFA*. El AFN resultante (Figura 1) tiene 14 estados, donde q_0 es el estado inicial y q_1 el único estado final. Las transiciones λ se deben a la construcción de la unión, la cerradura de Kleene y la concatenación.

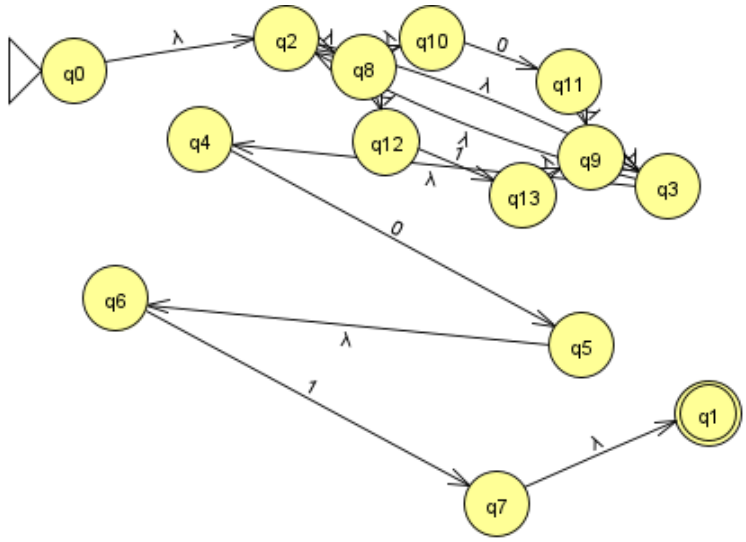


Figura 1: AFN generado por JFLAP a partir de la expresión $(0 + 1)^*01$

2.3. Predicciones

Antes de ejecutar el autómata, se predice el resultado de cada cadena. Como la expresión exige que toda cadena aceptada termine en el sufijo `01`, basta con revisar sus dos últimos símbolos (Tabla 2).

Tabla 2: Predicción de aceptación para la expresión regular $(0+1)^*01$

Cadena	Predicción	Justificación
`01`	Aceptada	$(0+1)^*$ genera λ y la cadena coincide exactamente con el sufijo `01`.
`101`	Aceptada	$(0+1)^*$ genera `1` y la cadena termina en `01`.
`1101`	Aceptada	$(0+1)^*$ genera `11` y la cadena termina en `01`.
`0001`	Aceptada	$(0+1)^*$ genera `00` y la cadena termina en `01`.
`10`	Rechazada	Termina en `10`; el último símbolo es `0` y no `1`.
`111`	Rechazada	No contiene ningún `0`, por lo que no puede terminar en `01`.
`0100`	Rechazada	Contiene `01`, pero no al final; termina en `00`.
`10101`	Aceptada	$(0+1)^*$ genera `101` y la cadena termina en `01`.

2.4. Ejecución en JFLAP y comparación

Las cadenas se ejecutaron sobre el AFN mediante la opción *Input* → *Multiple Run* de JFLAP. Los resultados obtenidos se muestran en la Figura 2.

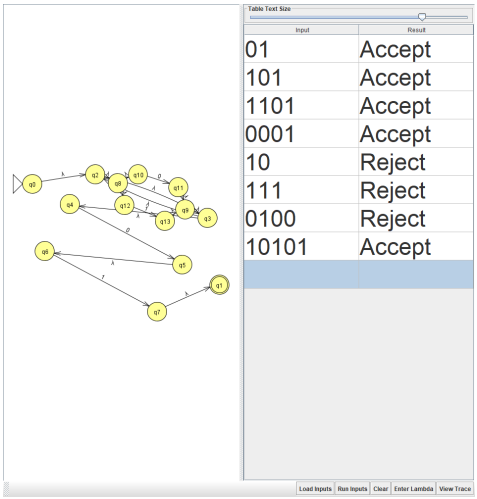


Figura 2: Resultados de la ejecución múltiple en JFLAP para la expresión $(0+1)^*01$

En la Tabla 3 se compara la predicción realizada antes de ejecutar el autómata con el resultado obtenido en JFLAP.

Tabla 3: Comparación entre la predicción y el resultado obtenido en JFLAP

Cadena	Predicción	Resultado JFLAP	¿Coinciden?
01	Aceptada	Aceptada	Sí
101	Aceptada	Aceptada	Sí
1101	Aceptada	Aceptada	Sí
0001	Aceptada	Aceptada	Sí
10	Rechazada	Rechazada	Sí
111	Rechazada	Rechazada	Sí
0100	Rechazada	Rechazada	Sí
10101	Aceptada	Aceptada	Sí

Las ocho predicciones coinciden con los resultados obtenidos en JFLAP. Esto confirma que el AFN generado reconoce exactamente las cadenas que terminan en 01, tal como describe la expresión regular.

2.5. Análisis de las cadenas 1101 y 0100

Para que una cadena pertenezca al lenguaje, debe poder dividirse en dos partes: un prefijo generado por $(0+1)^*$ y el sufijo obligatorio 01 al final.

Cadena 1101 (aceptada). La cadena se puede dividir como:

$$1101 = \underbrace{11}_{(0+1)^*} \cdot \underbrace{01}_{\text{sufijo}}$$

La parte $(0+1)^*$ genera el prefijo 11 (dos repeticiones de la unión, eligiendo 1 en ambas) y luego se lee el sufijo 01. En el AFN, esto corresponde a recorrer el ciclo de la cerradura de Kleene dos veces leyendo 1, salir del ciclo mediante transiciones λ, leer 0 y luego 1, y llegar al estado final q_1 justo al terminar la cadena. Como existe al menos un camino que termina en un estado de aceptación, la cadena es aceptada.

Cadena 0100 (rechazada). Los dos últimos símbolos de la cadena son 00, por lo que no existe ninguna división de la forma $x \cdot 01$ con $x \in \{0,1\}^*$. Aunque la cadena contiene la subcadena 01 en las posiciones 1 y 2, después de ella quedan los símbolos 00, y la expresión no permite ningún símbolo después del sufijo 01. En el AFN, si se lee 01 y se llega a q_1, todavía quedan símbolos por leer y q_1 no tiene transiciones de salida, por lo que ese camino se detiene sin consumir la cadena completa. Cualquier otro camino termina en un estado que no es de aceptación. Como ningún camino termina en q_1 al leer toda la cadena, la cadena es rechazada.

Conclusión. La diferencia entre ambas cadenas es la posición de la subcadena 01: en 1101 aparece al final, mientras que en 0100 aparece al inicio. El lenguaje no exige solo que la cadena contenga 01, sino que termine exactamente en 01.

3. Pregunta 2: Diseño de una expresión regular

Se considera el lenguaje:

$$L_2 = \{\, w \in \{a, b\}^* \mid w \text{ comienza con } a \text{ y termina con } b \,\}$$

3.1. Diseño de la expresión regular

Se tiene en cuenta que la cadena debe comenzar con a y terminar con b; esa es la restricción del lenguaje. Por lo tanto, la expresión tendrá una a al inicio, en el medio cualquier combinación posible de símbolos del alfabeto y una b al final. Así, la expresión regular diseñada es:

$$a(a + b)^* b$$

3.2. Justificación de la expresión

En la Tabla 4 se explica la función de cada parte de la expresión.

Tabla 4: Componentes de la expresión regular $a(a + b)^* b$ y su función

Parte	**Función**
a	Obliga a que la cadena comience con el símbolo `a`.
$(a + b)^*$	Permite cualquier secuencia intermedia de `a` y `b`, incluida la cadena vacía λ.
b	Obliga a que la cadena termine con el símbolo `b`.

Como la parte intermedia $(a + b)^*$ puede generar la cadena vacía, la cadena de longitud mínima del lenguaje es `ab`. Las cadenas de un solo símbolo, como `a` o `b`, no pertenecen a L_2, ya que un mismo símbolo no puede ser a la vez la `a` inicial y la `b` final.

3.3. AFN generado por el diseño planteado

La expresión se ingresó en JFLAP mediante la opción *Regular Expression* y se convirtió con *Convert* $\rightarrow$ *Convert to NFA*. El AFN resultante se muestra en la Figura 3, donde q_0 es el estado inicial y q_1 el estado final. Las transiciones λ provienen de la construcción de la concatenación, la unión $(a + b)$ y la cerradura de Kleene.

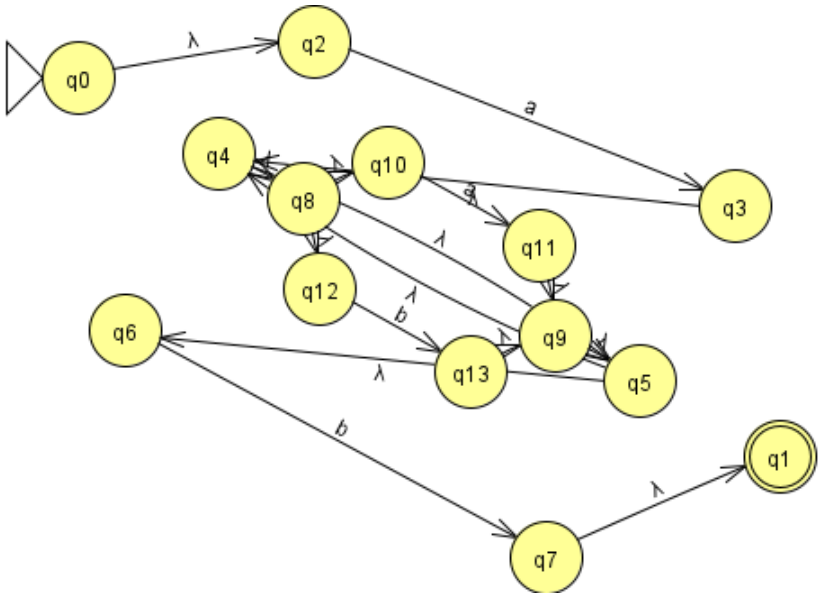


Figura 3: AFN generado por JFLAP a partir de la expresión $a(a+b)^*b$

3.4. Resultados esperados y obtenidos en JFLAP

Se definieron diez cadenas de prueba, incluyendo cadenas aceptadas, rechazadas, la cadena de longitud mínima y casos límite que cumplen solo una de las dos condiciones del lenguaje. Las cadenas se ejecutaron en JFLAP mediante *Input* → *Multiple Run* (Figura 4) y los resultados se resumen en la Tabla 5.

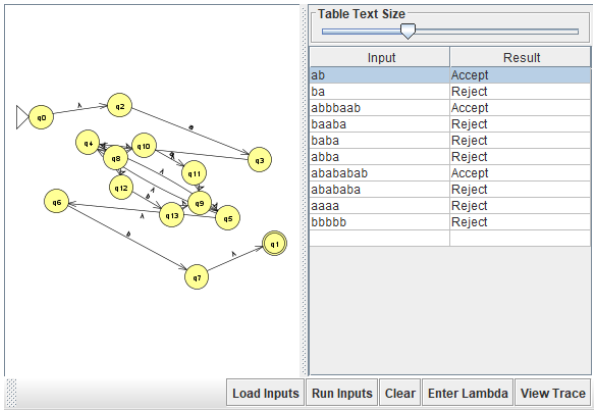


Figura 4: Resultados de la ejecución múltiple en JFLAP para la expresión $a(a+b)^*b$

Tabla 5: Casos de prueba para la expresión regular $a(a+b)^*b$

Cadena	**Resultado esperado**	**Resultado JFLAP**	**Observación**
`ab`	Aceptada	Aceptada	Cadena de longitud mínima.
`ba`	Rechazada	Rechazada	Invierte ambas condiciones.
`abbbaab`	Aceptada	Aceptada	Comienza con `a` y termina con `b`.
`baaba`	Rechazada	Rechazada	No comienza con `a` ni termina con `b`.
`baba`	Rechazada	Rechazada	No comienza con `a` ni termina con `b`.
`abba`	Rechazada	Rechazada	Comienza con `a`, pero termina con `a`.
`abababab`	Aceptada	Aceptada	Comienza con `a` y termina con `b`.
`abababa`	Rechazada	Rechazada	Comienza con `a`, pero termina con `a`.
`aaaa`	Rechazada	Rechazada	Caso límite: comienza con `a`, pero no termina con `b`.
`bbbbb`	Rechazada	Rechazada	Caso límite: termina con `b`, pero no comienza con `a`.

Los diez resultados obtenidos en JFLAP coinciden con los esperados. Los casos límite `aaaa` y `bbbbb` confirman que la expresión exige ambas condiciones a la vez, ya que una cadena que cumple solo una de ellas es rechazada. Además, la cadena `ab` demuestra que la parte intermedia $(a+b)^*$ puede ser vacía.

4. Pregunta 3: Conversión ER → AFN → AFD

Se considera el lenguaje:

$$L_3 = \{\, w \in \{0,1\}^* \mid w \text{ contiene la subcadena } 101 \,\}$$

4.1. Diseño de la expresión regular

Como la subcadena `101` puede aparecer en cualquier posición, la cadena se divide en tres partes: un prefijo cualquiera, la subcadena obligatoria y un sufijo cualquiera. Así, la expresión regular diseñada es:

$$(0+1)^*101(0+1)^*$$

Tabla 6: Componentes de la expresión regular $(0+1)^*101(0+1)^*$

Parte	Función
$(0+1)^*$	Permite cualquier secuencia antes de la subcadena, incluida la cadena vacía λ.
101	Obliga a que la cadena contenga la subcadena `101`.
$(0+1)^*$	Permite cualquier secuencia después de la subcadena, incluida la cadena vacía λ.

Como ambos extremos pueden ser vacíos, la cadena de longitud mínima del lenguaje es `101`.

4.2. Conversión de la expresión regular a AFN

La expresión se ingresó en JFLAP mediante la opción *Regular Expression* y se convirtió con *Convert* → *Convert to NFA*. El AFN resultante se muestra en la Figura 5, donde q_0 es el estado inicial y q_1 el único estado final. Las transiciones λ provienen de la construcción de las dos uniones $(0+1)$, las dos cerraduras de Kleene y la concatenación de las cinco partes de la expresión.

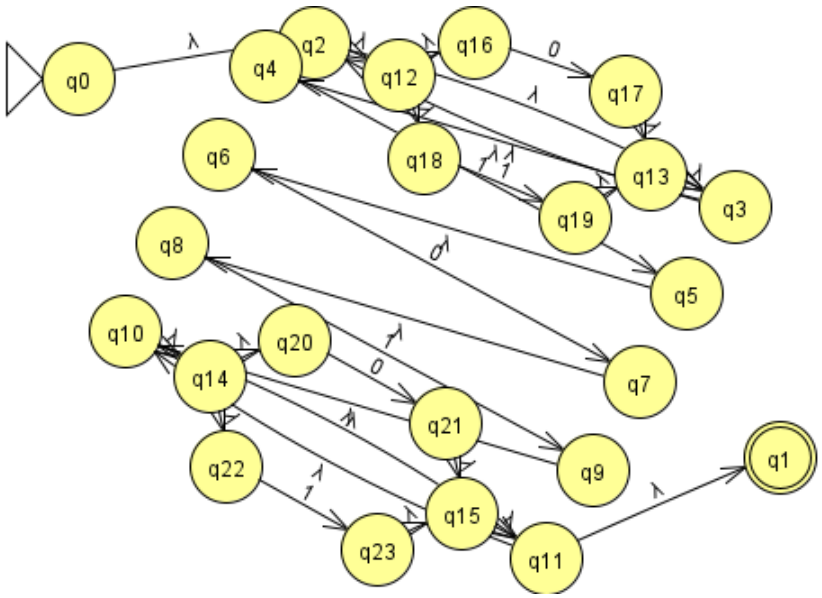


Figura 5: AFN generado por JFLAP a partir de la expresión $(0+1)^*101(0+1)^*$

4.3. Conversión de AFN a AFD

A partir del AFN se aplicó *Convert* → *Convert to DFA*, opción *Complete*. JFLAP utiliza la construcción por subconjuntos: cada estado del AFD representa un conjunto de estados del AFN, que se muestra en la etiqueta bajo cada estado. El AFD resultante se muestra en la Figura 6.

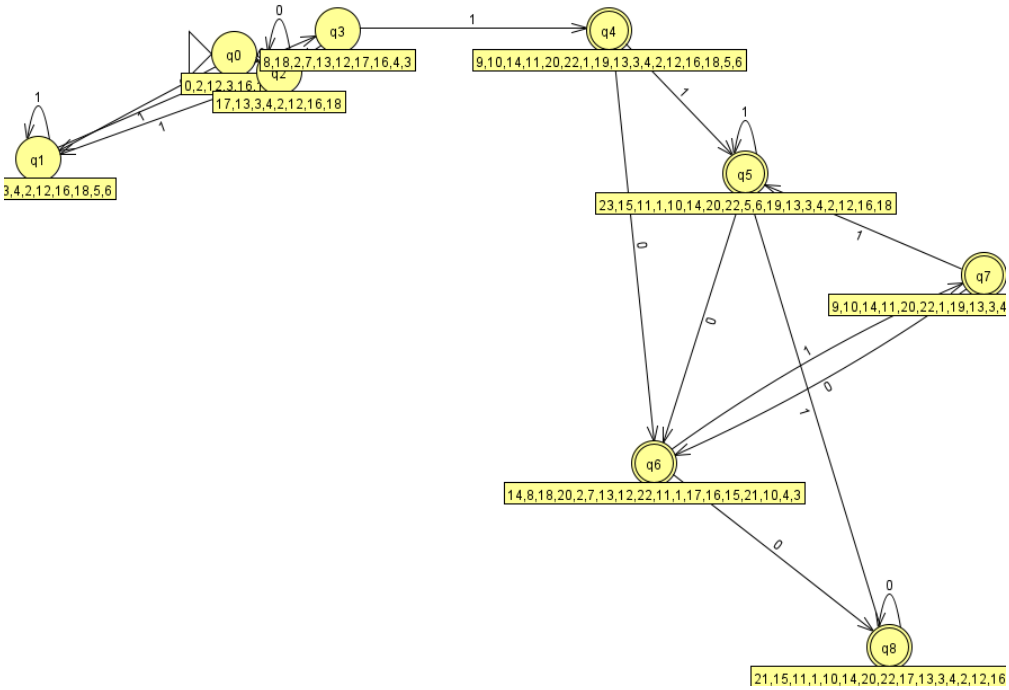


Figura 6: AFD obtenido a partir del AFN de la expresión $(0+1)^*101(0+1)^*$

4.4. Características de ambos autómatas

En la Tabla 7 se comparan las características de ambos autómatas.

Tabla 7: Comparación de características entre el AFN y el AFD

Característica	**AFN**	**AFD**
N.º de estados	24	9
N.º de transiciones	29	18
Transiciones λ	Sí (22)	No
¿Es determinista?	No	Sí

El AFN generado por JFLAP contiene 24 estados y un gran número de transiciones λ. Al aplicar la construcción por subconjuntos, el AFD resultante se reduce a 9 estados sin transiciones λ, con exactamente una transición por cada símbolo del alfabeto desde cada estado. Los estados q_0 a q_3 representan las situaciones en que aún no se ha leído la subcadena `101`, mientras que los estados q_4 a q_8 son de aceptación y corresponden a cadenas que ya la contienen; desde ellos todas las transiciones llevan a otro estado de aceptación, pues una vez leída la subcadena ningún símbolo posterior puede sacar a la cadena del lenguaje. Este AFD no es mínimo: el AFD mínimo para L_3 tiene 4 estados, ya que todos los estados de aceptación son equivalentes entre sí.

4.5. Ejecución de cadenas en AFN y AFD

Las cadenas indicadas se ejecutaron en ambos autómatas mediante *Input* → *Multiple Run* de JFLAP. La Figura 7 muestra los resultados en el AFN y la Figura 8 los resultados en el AFD.

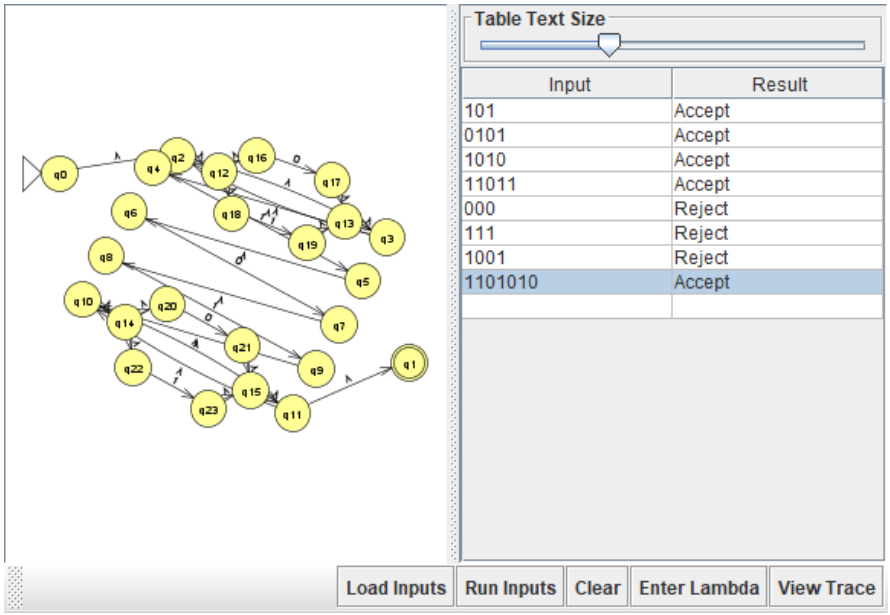


Figura 7: Resultados de la ejecución múltiple en el AFN de la expresión $(0+1)^*101(0+1)^*$

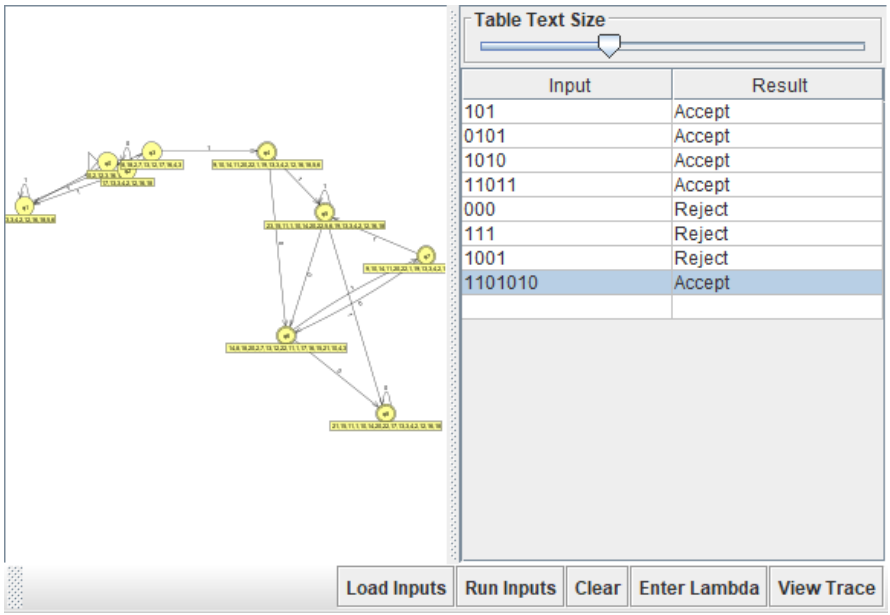


Figura 8: Resultados de la ejecución múltiple en el AFD obtenido por construcción por subconjuntos

En la Tabla 8 se resumen y comparan los resultados obtenidos en ambos autómatas.

Tabla 8: Resultados de las cadenas en el AFN y en el AFD

Cadena	AFN	AFD	¿Mismo resultado?
101	Aceptada	Aceptada	Sí
0101	Aceptada	Aceptada	Sí
1010	Aceptada	Aceptada	Sí
11011	Aceptada	Aceptada	Sí
000	Rechazada	Rechazada	Sí
111	Rechazada	Rechazada	Sí
1001	Rechazada	Rechazada	Sí
1101010	Aceptada	Aceptada	Sí

Las ocho cadenas obtuvieron el mismo resultado en ambos autómatas. Las cadenas aceptadas contienen la subcadena 101 en distintas posiciones: completa (101), al final (0101), al inicio (1010), en el medio (11011) y repetida (1101010). Las rechazadas no la contienen: 000 no tiene ningún 1, 111 no tiene ningún 0, y en 1001 los dos 1 están separados por dos ceros en lugar de uno.

4.6. ¿El AFN y el AFD reconocen lenguajes diferentes?

No. Ambos autómatas reconocen exactamente el mismo lenguaje L_3. El AFD se obtuvo mediante la construcción por subconjuntos, en la que cada estado del AFD representa el conjunto de todos los estados en que podría encontrarse el AFN después de leer un mismo prefijo, considerando sus λ-clausuras. Un estado del AFD es de aceptación si y solo si contiene al menos un estado final del AFN; por ejemplo, todos los estados finales q_4 a q_8 del AFD incluyen en su etiqueta el estado 1, que corresponde al estado final q_1 del AFN. Por lo tanto, para toda cadena w, el AFD acepta w exactamente cuando existe algún camino del AFN que acepta w.

Las diferencias entre ambos son estructurales y no de capacidad de reconocimiento: el AFN tiene más estados, transiciones λ y puede seguir varios caminos para una misma cadena, mientras que el AFD tiene un único camino posible para cada cadena y ninguna transición λ. Los resultados de la Tabla 8 confirman experimentalmente esta equivalencia, ya que las ocho cadenas obtuvieron el mismo resultado en ambos autómatas.

5. Pregunta 4: Problema de diseño y validación

Se considera el alfabeto $\Sigma = \{a, b\}$ y el lenguaje de las cadenas que cumplen simultáneamente dos condiciones: contienen al menos una `a` y terminan en `bb`.

$$L_4 = \{\, w \in \{a,b\}^* \mid w \text{ contiene al menos una } a \text{ y } w \text{ termina en } bb \,\}$$

5.1. Diseño de la expresión regular

Las dos condiciones se garantizan por separado y luego se concatenan. El sufijo `bb` se escribe al final de la expresión, lo que obliga a que toda cadena termine en `bb`. La condición de contener al menos una `a` se obliga escribiendo una `a` explícita, rodeada de cadenas arbitrarias, antes de ese sufijo. Así, la expresión regular diseñada es:

$$(a+b)^*\, a\, (a+b)^*\, bb$$

Tabla 9: Componentes de la expresión regular $(a+b)^*a(a+b)^*bb$

Parte	Función
$(a+b)^*$ (primera)	Permite cualquier secuencia antes de la `a` obligatoria, incluida la cadena vacía λ.
a	Garantiza que la cadena contiene al menos una `a`.
$(a+b)^*$ (segunda)	Permite cualquier secuencia entre la `a` obligatoria y el sufijo, incluida λ.
bb	Obliga a que la cadena termine en `bb`.

5.2. Estrategia utilizada

La estrategia consiste en ubicar la `a` obligatoria *antes* del sufijo `bb`. Como una cadena que termina en `bb` tiene sus dos últimos símbolos fijos, la `a` solo puede encontrarse en las posiciones anteriores, por lo que la expresión no necesita contemplar otra ubicación. De esta forma ambas condiciones se cumplen a la vez: la `a` aparece por construcción y el sufijo `bb` cierra la cadena.

Las cadenas de longitud mínima del lenguaje tienen tres símbolos y solo existe una: `abb`. Las cadenas `bb` (termina en `bb`, pero no tiene ninguna `a`) y `aaa` (tiene `a`, pero no termina en `bb`) son rechazadas porque cumplen solo una de las dos condiciones. Una expresión equivalente, más corta, es $b^*a(a+b)^*bb$, ya que el prefijo anterior a la primera `a` solo puede contener `b`; se eligió la primera forma por reflejar directamente la estructura prefijo–subcadena–sufijo.

5.3. Implementación en JFLAP

La expresión se ingresó en JFLAP mediante la opción *Regular Expression*, con la sintaxis compatible con la herramienta (el símbolo + para la unión y $*$ para la cerradura de Kleene). La expresión ingresada se muestra en la Figura 9.

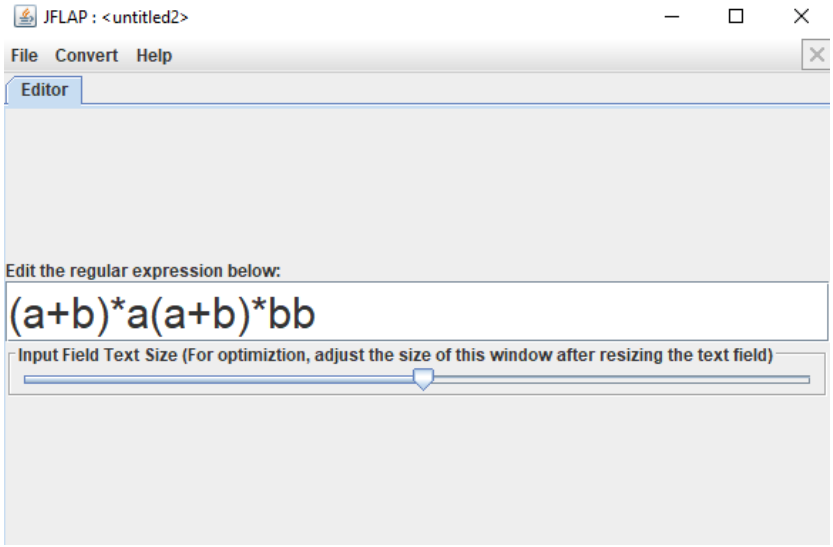


Figura 9: Expresión regular $(a+b)^*a(a+b)^*bb$ ingresada en JFLAP

5.4. Conversión ER → AFN → AFD

Se aplicó *Convert* → *Convert to NFA* para obtener el AFN (Figura 10) y, a partir de este, *Convert* → *Convert to DFA* para obtener el AFD (Figura 11).

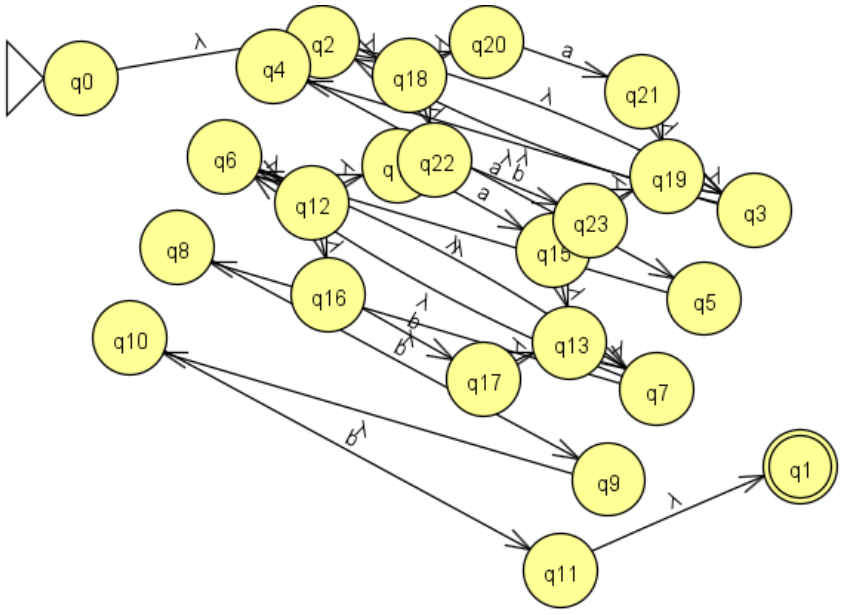


Figura 10: AFN generado por JFLAP a partir de la expresión $(a+b)^*a(a+b)^*bb$

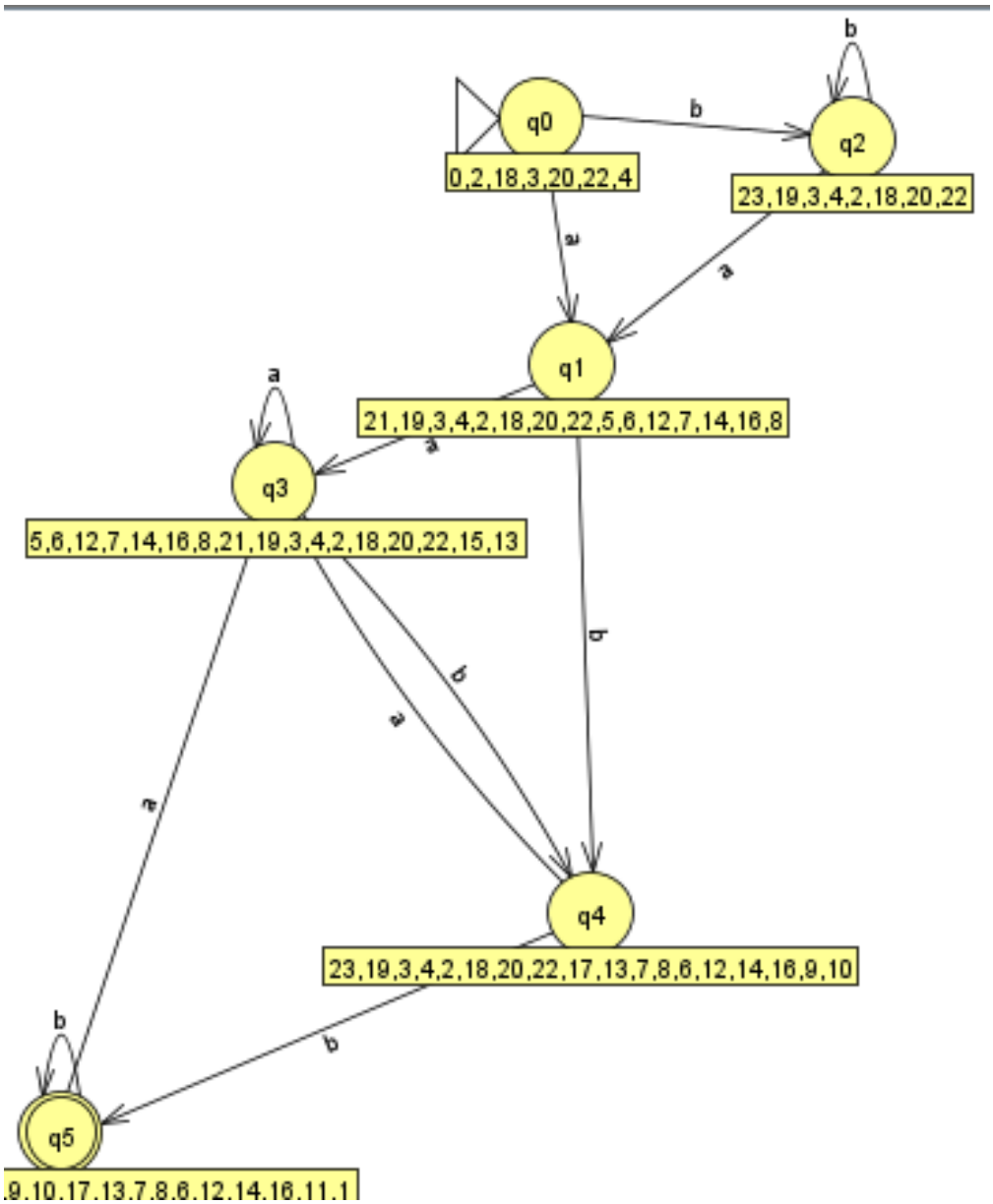


Figura 11: AFD obtenido a partir del AFN de la expresión $(a+b)^*a(a+b)^*bb$

En la Tabla 10 se comparan las características de ambos autómatas.

Tabla 10: Comparación de características entre el AFN y el AFD de L_4

Característica	**AFN**	**AFD**
N.º de estados	24	6
N.º de transiciones	29	12
Transiciones λ	Sí (22)	No
¿Es determinista?	No	Sí

El AFN tiene la misma estructura que el de la Pregunta 3, porque la expresión también se compone de cinco partes concatenadas y dos cerraduras de Kleene sobre una unión. El AFD resultante tiene 6 estados, con exactamente una transición por cada símbolo desde cada estado. Cada estado del AFD cumple un rol claro: q_0 es el estado inicial; q_2 se alcanza al leer solo `b` (aún no hay ninguna `a`); q_1 y q_3 representan que ya se leyó una `a` y la última lectura fue una `a`; q_4 indica que ya hay una `a` y la cadena termina en una sola `b`; y q_5 es el único estado de aceptación, que se alcanza cuando ya hay una `a` y la cadena termina en `bb`. Este AFD no es mínimo: q_0 y q_2 son equivalentes (ambos significan "todavía sin `a`") y también lo son q_1 y q_3, por lo que el AFD mínimo tiene 4 estados.

5.5. Casos de prueba

Se definieron 14 cadenas: 5 aceptadas, 5 rechazadas y 4 casos límite definidos por el grupo (la cadena vacía, una cadena que cumple solo la condición del sufijo, una cadena que contiene `bb` pero no al final y una cadena aceptada con más de dos `b` finales). Las cadenas se ejecutaron en el AFN y en el AFD mediante *Input* → *Multiple Run* (Figuras 12 y 13) y los resultados se resumen en la Tabla 11. En ambas figuras, la última fila sin texto corresponde a la cadena vacía λ, ya que JFLAP no muestra un símbolo para ella.

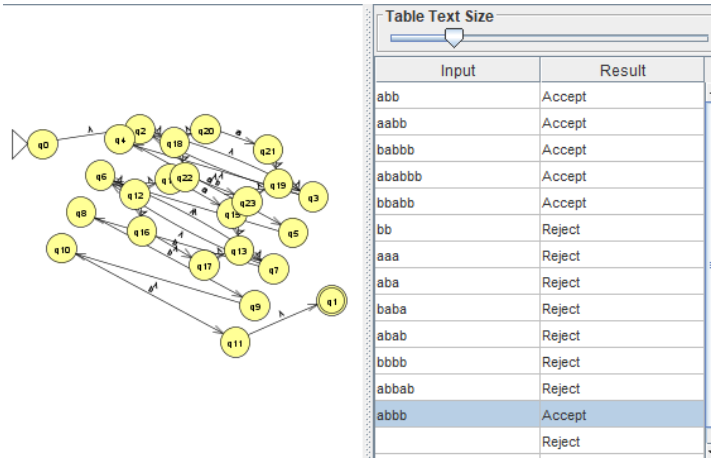


Figura 12: Resultados de la ejecución múltiple en el AFN de la expresión $(a+b)^*a(a+b)^*bb$

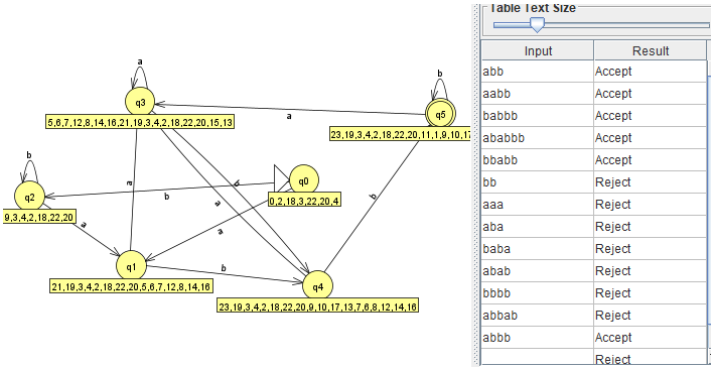


Figura 13: Resultados de la ejecución múltiple en el AFD de la expresión $(a+b)^*a(a+b)^*bb$

Tabla 11: Casos de prueba para el lenguaje L_4

Cadena	Esperado	AFN	AFD	Observación
`abb`	Aceptada	Aceptada	Aceptada	Cadena de longitud mínima.
`aabb`	Aceptada	Aceptada	Aceptada	Dos `a` seguidas y sufijo `bb`.
`babbb`	Aceptada	Aceptada	Aceptada	Ejemplo del enunciado.
`ababbb`	Aceptada	Aceptada	Aceptada	Ejemplo del enunciado.
`bbabb`	Aceptada	Aceptada	Aceptada	Contiene `bb` al inicio y al final.
`bb`	Rechazada	Rechazada	Rechazada	Termina en `bb`, pero no tiene `a`.
`aaa`	Rechazada	Rechazada	Rechazada	Tiene `a`, pero no termina en `bb`.
`aba`	Rechazada	Rechazada	Rechazada	Termina en `a`.
`baba`	Rechazada	Rechazada	Rechazada	Termina en `a`.
`abab`	Rechazada	Rechazada	Rechazada	Termina en `ab`, no en `bb`.
λ	Rechazada	Rechazada	Rechazada	Caso límite: cadena vacía.
`bbbb`	Rechazada	Rechazada	Rechazada	Caso límite: cumple el sufijo, pero no tiene `a`.
`abbab`	Rechazada	Rechazada	Rechazada	Caso límite: tiene `a` y `bb`, pero `bb` no es sufijo.
`abbb`	Aceptada	Aceptada	Aceptada	Caso límite: más de dos `b` finales.

Los resultados del AFN y del AFD coinciden con los esperados en las 14 cadenas. Los casos `bb` y `bbbb` muestran que cumplir solo el sufijo no basta, y `aaa` y `abbab` que tener una `a` no basta si `bb` no cierra la cadena. Por lo tanto, la expresión exige ambas condiciones simultáneamente.

5.6. Reflexión

Las tres representaciones tienen la misma capacidad de reconocimiento: describen exactamente la clase de los lenguajes regulares (teorema de Kleene). La expresión regular es una descripción declarativa del lenguaje, mientras que el AFN y el AFD son modelos operacionales. En este laboratorio, la expresión, el AFN de 24 estados y el AFD de 6 estados aceptaron y rechazaron las mismas 14 cadenas. Las transiciones λ y el no determinismo del AFN facilitan la construcción, pero no aumentan su poder: la construcción por subconjuntos transforma cualquier AFN en un AFD equivalente y la construcción de Thompson transforma cualquier expresión en un AFN. Por lo tanto, son formas distintas de representar una misma clase de lenguajes; difieren en tamaño, comodidad y forma de ejecución, pero no en los lenguajes que pueden reconocer.

6. Conclusión

El laboratorio permitió comprobar que una expresión regular, un AFN y un AFD son tres representaciones equivalentes de un mismo lenguaje regular. En las cuatro preguntas, las cadenas ejecutadas en JFLAP coincidieron con lo esperado a partir de la expresión regular, y en las Preguntas 3 y 4 el AFN y el AFD entregaron el mismo resultado para todas las cadenas probadas. Diseñar la expresión por partes (prefijo, subcadena obligatoria y sufijo) resultó una estrategia útil para garantizar condiciones simultáneas, como contener una `a` y terminar en `bb`.

Respecto a las diferencias entre autómatas, el AFN generado por JFLAP es grande y contiene muchas transiciones λ (24 estados y 22 transiciones λ en las Preguntas 3 y 4), mientras que el AFD obtenido por subconjuntos es más pequeño (9 y 6 estados) y no tiene transiciones λ, con un único camino posible por cadena; ambos AFD pueden reducirse aún más a un AFD mínimo de 4 estados. JFLAP resultó una herramienta útil para construir, convertir y validar lenguajes regulares, ya que permite ejecutar varias cadenas a la vez y observar directamente la estructura de cada autómata.

7. Archivos adjuntos

Junto con este informe se entregan los siguientes archivos de JFLAP (.jff), que corresponden a los autómatas presentados en cada pregunta:

- `P1_AFN.jff`
- `P2_AFN.jff`
- `P3_AFN.jff` y `P3_AFD.jff`
- `P4_AFN.jff` y `P4_AFD.jff`

Enlace al repositorio (GitHub o Drive) con el informe en PDF y los archivos .jff:

- Drive
- Github